

Strings and Character Array Manipulation in C

📌 Definition to Remember

In C, a string is a 1D array of characters terminated by a special null character (" " with ASCII value 0), manipulated via library functions.

1. Concept Overview & Fundamentals 🧠

Strings and Character Array Manipulation in C forms a core cornerstone of computer systems programming, hardware-level software engineering, and competitive problem solving.

In the C execution model:

- Code is compiled directly into native machine instructions without virtual machine overhead.
- Memory is strictly partitioned into **Code/Text Segment**, **Data Segment (Initialized/BSS)**, **Stack**, and **Heap**.
- Direct pointer and register manipulation provides deterministic performance critical for operating systems, embedded hardware, and database kernels.

2. Technical Architecture & Memory Mechanics ⚙️

Memory Layout & Storage Organization

```
+-----+
|                STACK SEGMENT                |
| - Local variables, function call stack frames, parameters |
| - High memory growing downwards towards Heap |
+-----+
|
| v
| ^
|
+-----+
|                HEAP SEGMENT                |
| - Dynamic memory allocations via malloc(), calloc() |
| - Low memory growing upwards towards Stack |
+-----+
|                INITIALIZED DATA SEGMENT    |
| - Global and static variables with non-zero initial values |
+-----+
|                BSS SEGMENT (UNINITIALIZED)  |
| - Global and static variables initialized to zero |
+-----+
|                CODE / TEXT SEGMENT          |
| - Read-only executable binary instructions |
+-----+
```

Core Architecture Breakdown

Component / Concept	Technical Specification	Operational Characteristics	Performance / Complexity
Storage Allocation	Stack vs Heap vs Data Segment	Stack is ultra-fast LIFO; Heap offers dynamic scalability	Stack: O(1) push/pop; Heap: O(1) alloc
Type Checking	Static Typing	Compiler verifies types during compilation	Zero runtime type penalty
Addressing Model	Byte-addressable physical RAM	Pointer stores hexadecimal byte memory address	Direct memory dereference (O(1))
Execution Flow	Procedural / Sequential	Controlled via jumps, branches, loops, and call stacks	Deterministic instruction cycles

3. Core Syntax & Implementation Patterns

Essential Idioms & Header Declarations

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

// Core standard implementation pattern
int main(void) {
    // Explicit initialization
    int status = 0;

    // Core logic execution
    printf("Executing C Module: Strings\n");

    return status;
}
```

4. Real-World Applications & System Engineering

- **Operating System Kernels:** Linux, UNIX, and Windows NT kernels utilize C for drivers, process scheduling, and memory paging.
- **Embedded & IoT Devices:** Microcontrollers with constrained RAM (8KB-64KB) require C's predictable footprint.
- **Database Engine Kernels:** High-throughput storage engines (PostgreSQL, SQLite, MySQL InnoDB) rely on low-level buffer pool managers written in C.
- **Game Engine Foundations:** Graphics rasterization, physics engines, and GPU interfaces utilize C/C++ memory arrays for zero-overhead computation.

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: break keyword loop

Problem Description & Objective:

Implementation of break keyword loop

C Code Implementation:

```
/*
Create a program using break to read inputs
from the user in a loop and break the loop
if a specific keyword (like "exit") is entered.
*/

#include <stdio.h>
#include <string.h>

int main() {
    /*
    * Logic:
    * 1. Uses an infinite while(1) loop to continuously request text input from the user.
    * 2. Uses %99s inside scanf to safely read input without exceeding the 100-character buffer
    capacity.
    * 3. Uses strcmp() to compare the user input with the keyword "exit".
    * 4. If the input is "exit", executes a break statement to exit the loop immediately.
    * 5. Otherwise, prints the user input and continues to the next loop iteration.
    */
    char text[100];

    while(1) {

        printf("Enter text (type 'exit' to quit): ");
        scanf("%99s", text);

        // Check exit condition
        if(strcmp(text, "exit") == 0) {
            break;
        }

        printf("You entered: %s\n", text);
    }

    printf("Program Ended.\n");

    return 0;
}
```

Problem 2: date format

Problem Description & Objective:

Implementation of date format

C Code Implementation:

```
/*
Use printf with format specifiers to format
and print a date string (day, month, year).
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * - Goal: Use printf with format specifiers to format and print a date string (day, month, year).
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int day = 1;
    int month = 6;
    int year = 2026;

    printf("Formatted Date: %02d/%02d/%d\n",
           day, month, year);

    return 0;
}
```

Problem 3: fgets puts

Problem Description & Objective:

Implementation of fgets puts

C Code Implementation:

```
/*
Read a line of text from the user using fgets
and then print it using puts.
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * - Goal: Read a line of text from the user using fgets and then print it using puts.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    char text[100];

    printf("Enter a line of text:\n");

    fgets(text, sizeof(text), stdin);

    printf("\nYou entered:\n");

    puts(text);

    return 0;
}
```

Problem 4: login system

Problem Description & Objective:

Implementation of login system

C Code Implementation:

```

/*
Create a simple text-based user login system
that compares a stored password string using strcmp.
*/

#include <stdio.h>
#include <string.h>

int main() {
    /*
    * Logic:
    * 1. Allocates a buffer of 100 bytes for the user password input.
    * 2. Reads input securely using a limit of 99 characters (%99s) to prevent buffer overflows.
    * 3. Uses strcmp() to check if the user input matches the stored password.
    * 4. If strcmp returns 0 (exact match), displays success; otherwise displays incorrect password.
    */
    char storedPassword[] = "admin123";
    char userPassword[100];

    printf("Enter password: ");
    scanf("%99s", userPassword);

    if(strcmp(storedPassword, userPassword) == 0) {
        printf("Login Successful!\n");
    } else {
        printf("Incorrect Password!\n");
    }

    return 0;
}

```

Problem 5: password checker

Problem Description & Objective:

Implementation of password checker

C Code Implementation:

```

/*
Create a program using do-while to find
password checker until a valid password is entered.
*/

#include <stdio.h>
#include <string.h>

int main() {
    /*
    * Logic:
    * 1. Uses a do-while loop to ensure the user is prompted to enter a password at least once.
    * 2. Safely reads string inputs with %99s to protect the 100-character buffer from overflow.
    * 3. Uses strcmp() to compare the entered password with "admin123".
    * 4. If they do not match, displays an error message and continues looping.
    * 5. Terminates the loop and displays a success message once the correct password is provided.
    */
    char password[100];

    do {

        printf("Enter password: ");
        scanf("%99s", password);

        if(strcmp(password, "admin123") != 0) {
            printf("Invalid Password! Try Again.\n");
        }

    } while(strcmp(password, "admin123") != 0);

    printf("Valid Password Entered.\n");

    return 0;
}

```

Problem 6: reverse string function

Problem Description & Objective:

Implementation of reverse string function

C Code Implementation:

```

/*
Write a function that takes a string
and reverses it in place.
*/

#include <stdio.h>
#include <string.h>

// Function Definition
void reverseString(char str[]) {
    /*
    * Logic:
    * 1. Uses strchr to find the index of '\n' and replaces it with '\0' to safely
    *    strip trailing newlines left by fgets, preventing index out-of-bounds on empty strings.
    * 2. Uses a two-pointer approach with 'start' index (0) and 'end' index (strlen(str) - 1).
    * 3. Swaps characters at 'start' and 'end' using a temporary variable.
    * 4. Increments start and decrements end in a loop until the pointers meet, reversing the string in-
    place.
    */
    str[strchr(str, "\n")] = '\0';

    int start = 0;
    int end = strlen(str) - 1;
    char temp;

    while(start < end) {

        temp = str[start];
        str[start] = str[end];
        str[end] = temp;

        start++;
        end--;
    }
}

int main() {
    char str[100];

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    reverseString(str);

    printf("Reversed String: %s\n", str);

    return 0;
}

```

Problem 7: string palindrome

Problem Description & Objective:

Implementation of string palindrome

C Code Implementation:

```

/*
Create a program that checks if a given
string is a palindrome and outputs the result.
*/

#include <stdio.h>
#include <string.h>

int main() {
    /*
    * Logic:
    * - Goal: Create a program that checks if a given string is a palindrome and outputs the result.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    char str[100];
    int start = 0, end;
    int isPalindrome = 1;

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    // Remove newline character
    str[strcspn(str, "\n")] = '\0';

    end = strlen(str) - 1;

    while(start < end) {

        if(str[start] != str[end]) {
            isPalindrome = 0;
            break;
        }

        start++;
        end--;
    }

    if(isPalindrome == 1) {
        printf("String is Palindrome.\n");
    } else {
        printf("String is NOT Palindrome.\n");
    }

    return 0;
}

```

Problem 8: tic tac toe board

Problem Description & Objective:

Implementation of tic tac toe board

C Code Implementation:

```

/*
Use a 2-D character array to store and
display a tic-tac-toe board.
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * - Goal: Use a 2-D character array to store and display a tic-tac-toe board.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    char board[3][3] = {
        {'X', 'O', 'X'},
        {'O', 'X', 'O'},
        {'X', ' ', 'O'}
    };

    int i, j;

    printf("Tic-Tac-Toe Board:\n\n");

    for(i = 0; i < 3; i++) {

        for(j = 0; j < 3; j++) {
            printf(" %c ", board[i][j]);

            if(j < 2) {
                printf("|");
            }
        }

        printf("\n");

        if(i < 2) {
            printf("-----\n");
        }
    }

    return 0;
}

```

Problem 9: trim string

Problem Description & Objective:

Implementation of trim string

C Code Implementation:

```

/*
Implement a trim function that removes leading
and trailing spaces from a string.
*/

#include <stdio.h>
#include <string.h>
#include <ctype.h>

// Function Definition
void trim(char str[]) {
    /*
    * Logic:
    * 1. Strips trailing newlines securely using strcspn() to protect empty bounds.
    * 2. Finds the index of the first non-whitespace character by moving 'start' forward
    *    with isspace() (safely cast to unsigned char).
    * 3. Finds the index of the last non-whitespace character by moving 'end' backward.
    * 4. Shifts the trimmed substring (from 'start' to 'end') to index 0 of the buffer.
    * 5. Places the null terminator '\0' at the correct position (j) to truncate the string.
    */
    str[strcspn(str, "\n")] = '\0';

    int start = 0;
    int end = strlen(str) - 1;
    int i, j;

    // Find first non-space character
    while(isspace((unsigned char)str[start])) {
        start++;
    }

    // Find last non-space character
    while(end >= start && isspace((unsigned char)str[end])) {
        end--;
    }

    // Shift string to beginning
    for(i = start, j = 0; i <= end; i++, j++) {
        str[j] = str[i];
    }

    str[j] = '\0';
}

int main() {
    char str[100];

    printf("Enter a string with spaces:\n");
    fgets(str, sizeof(str), stdin);

    trim(str);

    printf("Trimmed String: '%s'\n", str);

    return 0;
}

```

Problem 10: uppercase string

Problem Description & Objective:

Implementation of uppercase string

C Code Implementation:

```
/*
Write a program to convert an input string
to uppercase.
*/

#include <stdio.h>
#include <ctype.h>

int main() {
    /*
    * Logic:
    * - Goal: Write a program to convert an input string to uppercase.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    char str[100];
    int i = 0;

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    while(str[i] != '\0') {

        str[i] = toupper(str[i]);
        i++;
    }

    printf("Uppercase String:\n%s", str);

    return 0;
}
```

6. Exam Scoring Points & Memory Revision

★ Must-Write Points

1. Null-terminator ('\0'): Marks the end of string buffer; array size must be at least length + 1.
2. Standard functions in : strlen(), strcpy(), strcat(), strcmp(), strrev().
3. Reading strings with spaces: Use fgets(str, size, stdin) instead of unsafe gets().
4. String comparison: strcmp(s1, s2) returns 0 if identical, <0 if s1 < s2, >0 if s1 > s2.
5. Character classification functions in : isalpha(), isdigit(), toupper(), tolower().

⚡ Quick Recall

`char array[]` -> `'\0'` Null Terminator -> `fgets(stdin)` -> `<string.h>` Ops -> Bounds Safety